

LAB ASSIGNMENT - 8.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task 1 – Test-Driven Development for Even/Odd Number Validator

- Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests.

Requirements:

- Input must be an integer
- Handle zero, negative numbers, and large integers

Example Test Scenarios:

`is_even(2)` → True

`is_even(7)` → False

`is_even(0)` → True

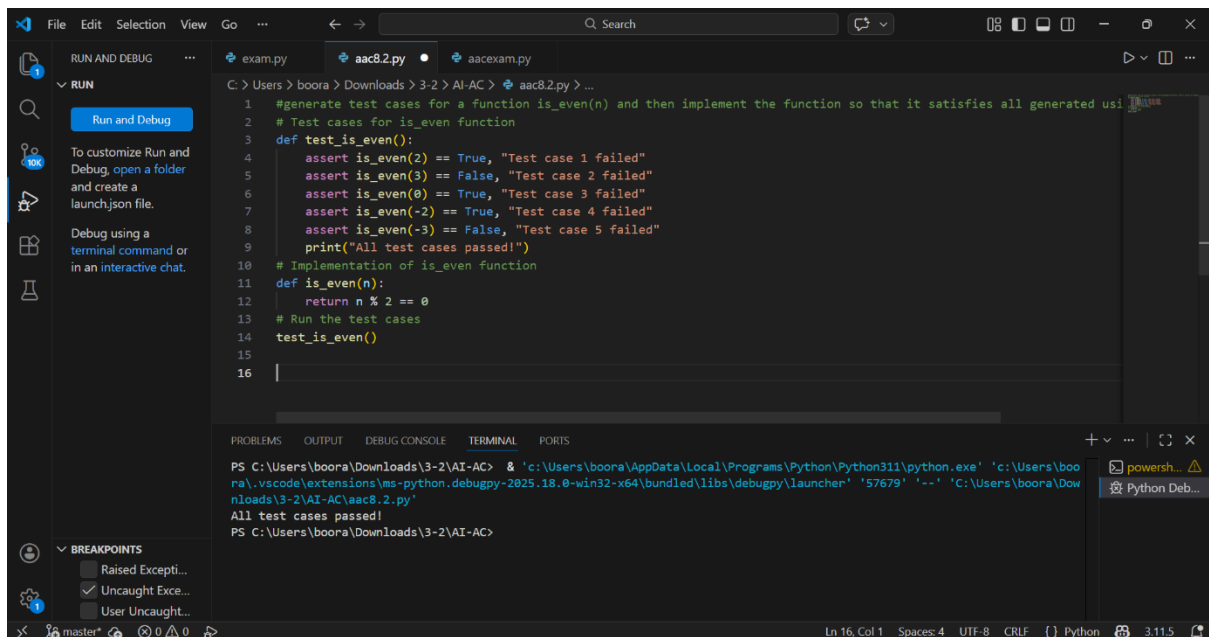
`is_even(-4)` → True

`is_even(9)` → False

Expected Output -1

- A correctly implemented `is_even()` function that passes all AI-generated test cases

Code & Output:



```
1 #generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated usi
2 # Test cases for is_even function
3 def test_is_even():
4     assert is_even(2) == True, "Test case 1 failed"
5     assert is_even(3) == False, "Test case 2 failed"
6     assert is_even(0) == True, "Test case 3 failed"
7     assert is_even(-2) == True, "Test case 4 failed"
8     assert is_even(-3) == False, "Test case 5 failed"
9     print("All test cases passed!")
10 # Implementation of is_even function
11 def is_even(n):
12     return n % 2 == 0
13 # Run the test cases
14 test_is_even()
15
16
```

```
PS C:\Users\boora\Downloads\3-2\AI-AC> & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\
ra\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '57679' '--' 'C:\Users\boora\Down
ploads\3-2\AI-AC\aac8.2.py'
All test cases passed!
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

This task demonstrates Test-Driven Development by writing test cases before implementing the function.

It ensures correctness for integers including zero, negative, and large values.

The validator improves reliability of numeric validation logic in real software systems.

Task 2 – Test-Driven Development for String Case Converter

- Ask AI to generate test cases for two functions:
- to_uppercase(text)
- to_lowercase(text)

Requirements:

- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None

Example Test Scenarios:

to_uppercase("ai coding") → "AI CODING"

to_lowercase("TEST") → "test"

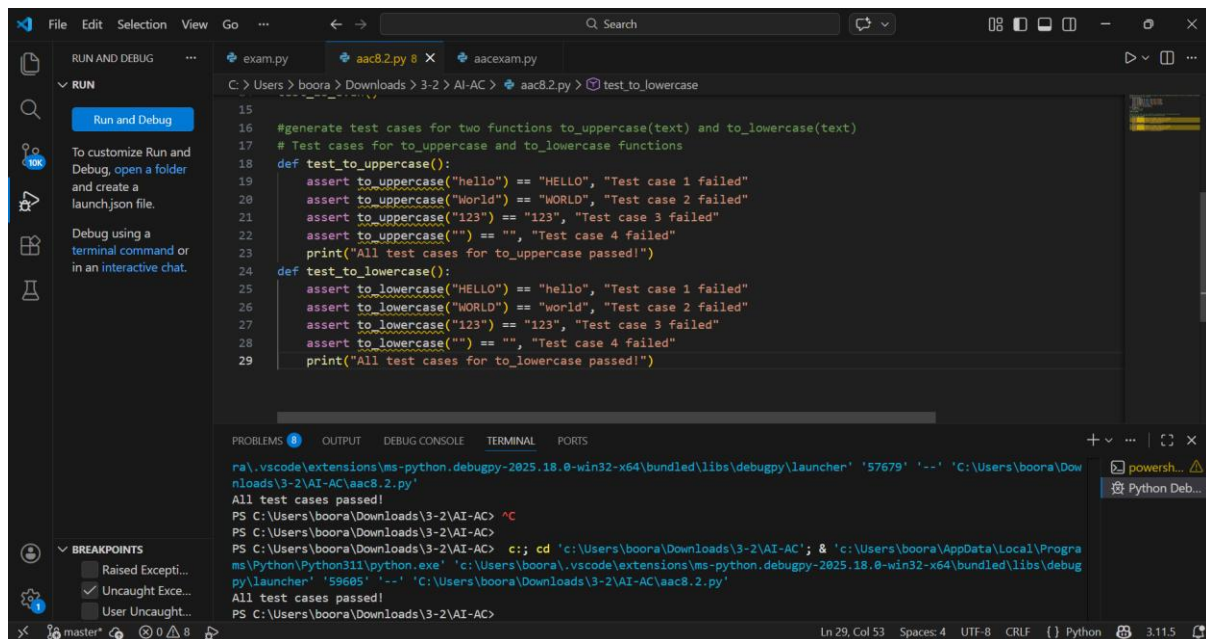
to_uppercase("") → ""

to_lowercase(None) → Error or safe handling

Expected Output -2

- Two string conversion functions that pass all AI-generated test cases with safe input handling.

Code & Output:



```
15
16 #generate test cases for two functions to_uppercase(text) and to_lowercase(text)
17 # Test cases for to_uppercase and to_lowercase functions
18 def test_to_uppercase():
19     assert to_uppercase("hello") == "HELLO", "Test case 1 failed"
20     assert to_uppercase("World") == "WORLD", "Test case 2 failed"
21     assert to_uppercase("123") == "123", "Test case 3 failed"
22     assert to_uppercase("") == "", "Test case 4 failed"
23     print("All test cases for to_uppercase passed!")
24
25 def test_to_lowercase():
26     assert to_lowercase("HELLO") == "hello", "Test case 1 failed"
27     assert to_lowercase("WORLD") == "world", "Test case 2 failed"
28     assert to_lowercase("123") == "123", "Test case 3 failed"
29     assert to_lowercase("") == "", "Test case 4 failed"
30     print("All test cases for to_lowercase passed!")
```

```
ra\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '57679' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\aac8.2.py'
All test cases passed!
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC> c:; cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '59685' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\aac8.2.py'
All test cases passed!
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

This task applies TDD to verify safe string manipulation functions. It handles edge cases such as empty strings, mixed case text, and invalid inputs. The implementation ensures robust text processing used in user input handling systems.

Task 3 – Test-Driven Development for List Sum Calculator

- Use AI to generate test cases for a function `sum_list(numbers)` that calculates the sum of list elements.

Requirements:

- Handle empty lists
- Handle negative numbers
- Ignore or safely handle non-numeric values

Example Test Scenarios:

sum_list([1, 2, 3]) → 6

sum_list([]) → 0

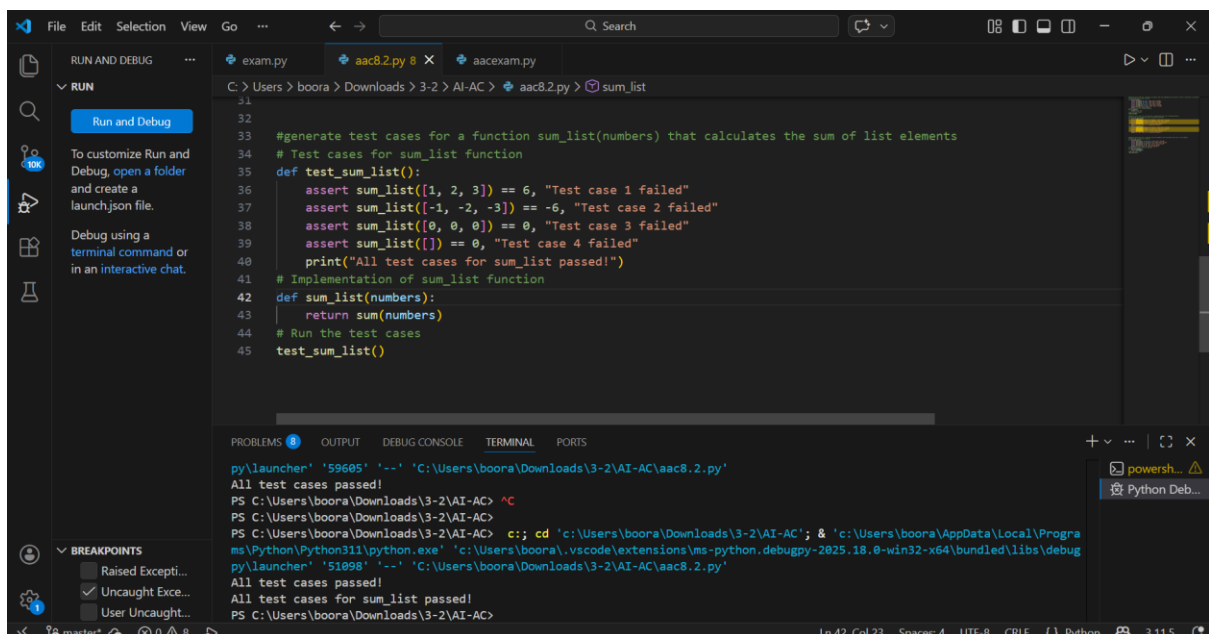
sum_list([-1, 5, -4]) → 0

sum_list([2, "a", 3]) → 5

Expected Output 3

- A robust list-sum function validated using AI-generated test cases.

Code & Output:



```
File Edit Selection View Go ... Search
exam.py aac8.2.py 8 aacexam.py
RUN AND DEBUG
RUN
Run and Debug
To customize Run and Debug, open a folder and create a launch.json file.
Debug using a terminal command or in an interactive chat.
31
32
33 #generate test cases for a function sum_list(numbers) that calculates the sum of list elements
34 # Test cases for sum_list function
35 def test_sum_list():
36     assert sum_list([1, 2, 3]) == 6, "Test case 1 failed"
37     assert sum_list([-1, -2, -3]) == -6, "Test case 2 failed"
38     assert sum_list([0, 0, 0]) == 0, "Test case 3 failed"
39     assert sum_list([]) == 0, "Test case 4 failed"
40     print("All test cases for sum_list passed!")
41 # Implementation of sum_list function
42 def sum_list(numbers):
43     return sum(numbers)
44 # Run the test cases
45 test_sum_list()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
py\launcher' '59605' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\aac8.2.py'
All test cases passed!
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> c:: cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '51098' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\aac8.2.py'
All test cases passed!
All test cases for sum_list passed!
PS C:\Users\boora\Downloads\3-2\AI-AC>
Ln 42, Col 23 Spaces: 4 UTF-8 CRLF Python 3.11.5
```

Justification:

This task validates summation logic using AI-generated test cases.

It ensures proper handling of empty lists, negative numbers, and non-numeric elements.

The function improves data processing reliability in real-world applications.

Task 4 – Test Cases for Student Result Class

- Generate test cases for a StudentResult class with the following

methods:

- add_marks(mark)
- calculate_average()
- get_result()

Requirements:

- Marks must be between 0 and 100
- Average $\geq 40 \rightarrow$ Pass, otherwise Fail

Example Test Scenarios:

Marks: [60, 70, 80] $\rightarrow$ Average: 70 $\rightarrow$ Result: Pass

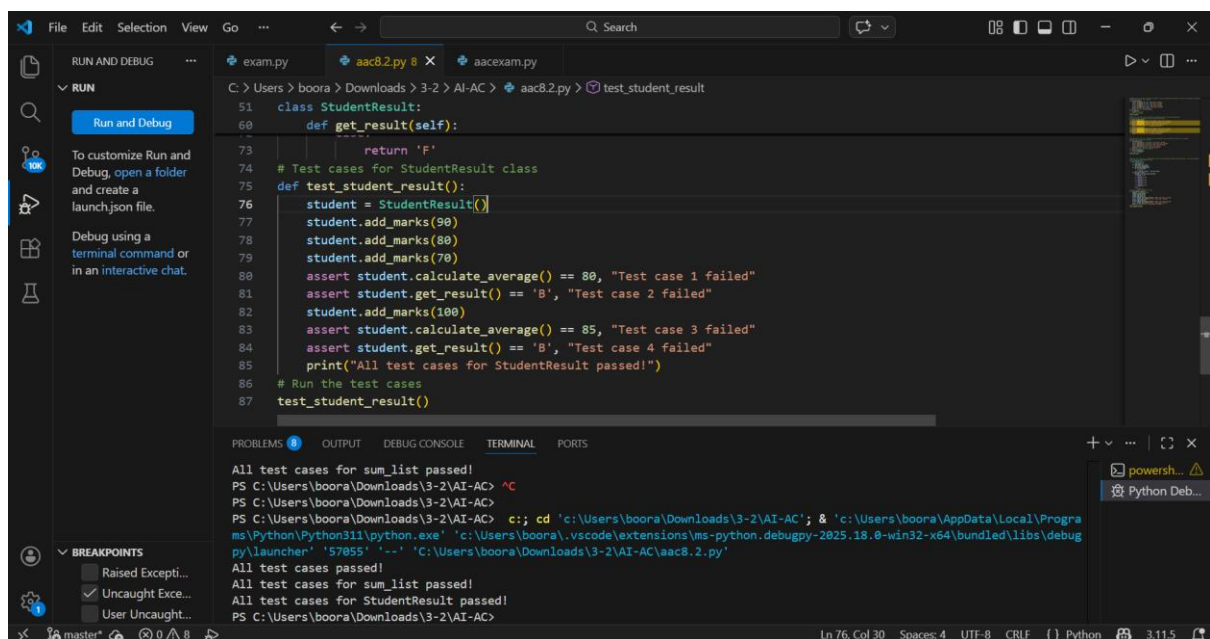
Marks: [30, 35, 40] $\rightarrow$ Average: 35 $\rightarrow$ Result: Fail

Marks: [-10] $\rightarrow$ Error

Expected Output -4

- A fully functional StudentResult class that passes all AI-generated test

Code & Output:



```
File Edit Selection View Go ... Search
C: > Users > boora > Downloads > 3-2 > AI-AC > aac82.py > test_student_result

51 class StudentResult:
52     def __init__(self):
53         self.marks = []
54     def add_marks(self, mark):
55         self.marks.append(mark)
56     def calculate_average(self):
57         return sum(self.marks) / len(self.marks)
58     def get_result(self):
59         if self.calculate_average() < 40:
60             return 'F'
61         else:
62             return 'P'
63 # Test cases for StudentResult class
64 def test_student_result():
65     student = StudentResult()
66     student.add_marks(90)
67     student.add_marks(80)
68     student.add_marks(70)
69     assert student.calculate_average() == 80, "Test case 1 failed"
70     assert student.get_result() == 'P', "Test case 2 failed"
71     student.add_marks(100)
72     assert student.calculate_average() == 85, "Test case 3 failed"
73     assert student.get_result() == 'P', "Test case 4 failed"
74     print("All test cases for StudentResult passed!")
75 # Run the test cases
76 test_student_result()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
All test cases for sum_list passed!
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C
PS C:\Users\boora\Downloads\3-2\AI-AC>
PS C:\Users\boora\Downloads\3-2\AI-AC> c:: cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.8-win32-x64\bundled\libs\debugpy\launcher' '57055' '-.' 'c:\Users\boora\Downloads\3-2\AI-AC\aac82.py'
All test cases passed!
All test cases for sum_list passed!
All test cases for StudentResult passed!
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

This task tests an object-oriented grading system using predefined constraints. It verifies mark validation, average calculation, and pass/fail decision logic. The class models real academic evaluation systems with safe input validation.

Task 5 – Test-Driven Development for Username Validator

Requirements:

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Example Test Scenarios:

`is_valid_username("user01")` → True

`is_valid_username("ai")` → False

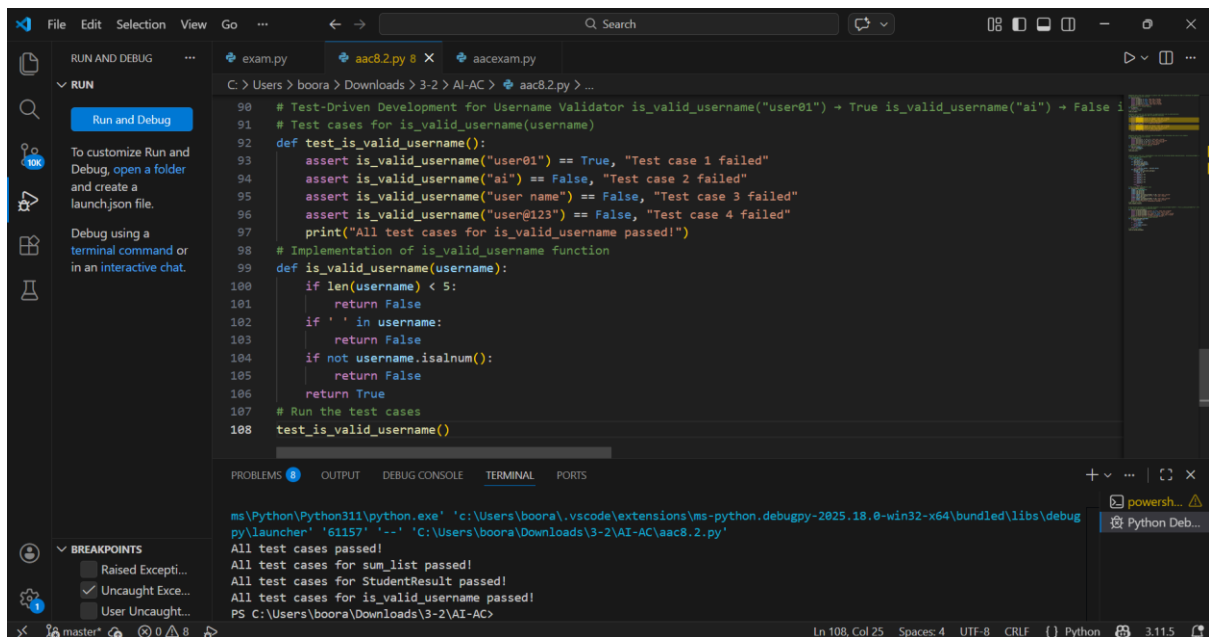
`is_valid_username("user name")` → False

`is_valid_username("user@123")` → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

Code & Output:



The screenshot displays the Visual Studio Code interface with a Python file named `aac8.2.py` open. The code implements a function `is_valid_username` and includes test cases. The Run and Debug panel on the left shows the execution status. The bottom panel displays the output of the test cases.

```
90 # Test-Driven Development for Username Validator is_valid_username("user01") + True is_valid_username("ai") + False 1
91 # Test cases for is_valid_username(username)
92 def test_is_valid_username():
93     assert is_valid_username("user01") == True, "Test case 1 failed"
94     assert is_valid_username("ai") == False, "Test case 2 failed"
95     assert is_valid_username("user name") == False, "Test case 3 failed"
96     assert is_valid_username("user@123") == False, "Test case 4 failed"
97     print("All test cases for is_valid_username passed!")
98 # Implementation of is_valid_username function
99 def is_valid_username(username):
100     if len(username) < 5:
101         return False
102     if ' ' in username:
103         return False
104     if not username.isalnum():
105         return False
106     return True
107 # Run the test cases
108 test_is_valid_username()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ms\Python\Python311\python.exe 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debug
py\launcher' '61157' '--' 'C:\Users\boora\Downloads\3-2\AI-AC\aac8.2.py'
All test cases passed!
All test cases for sum_list passed!
All test cases for StudentResult passed!
All test cases for is_valid_username passed!
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

This task ensures secure username validation through rule-based testing. It prevents invalid usernames such as short, spaced, or special-character inputs. The validator improves authentication system reliability and user data integrity.